Cloud Computing: Resilience & DevOps

Module 8: Resilience, Observability & DevOps

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-III





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Outline

1. High Availability & Resilience
2. Backup & Disaster Recovery
3. Monitoring & Observability
4. DevOps & Infrastructure as Code





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Outline

1. High Availability & Resilience
2. Backup & Disaster Recovery
3. Monitoring & Observability
4. DevOps & Infrastructure as Code





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Availability vs. Reliability

Availability

- System is up and running
- Measured as uptime percentage
- $99.9\% = 8.7$ hours down/year
- $99.99\% = 52$ min down/year

Reliability

- System works correctly
- Does it do what's expected?
- Can be available but unreliable
- Requires testing, validation

Both Matter

You need both high availability AND reliability for production systems.





Availability vs. Reliability

Availability

- System is up and running
- Measured as uptime percentage
- $99.9\% = 8.7$ hours down/year
- $99.99\% = 52$ min down/year

Reliability

- System works correctly
- Does it do what's expected?
- Can be available but unreliable
- Requires testing, validation



Both Matter

You need both high availability AND reliability for production systems

Availability vs. Reliability

Availability

- System is up and running
- Measured as uptime percentage
- $99.9\% = 8.7$ hours down/year
- $99.99\% = 52$ min down/year

Reliability

- System works correctly
- Does it do what's expected?
- Can be available but unreliable
- Requires testing, validation

Both Matter

You need both high availability AND reliability for production systems.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Achieving High Availability

Key Strategies

Redundancy at every layer: compute, storage, network, database.

Techniques:

- Multi-zone deployment: instances in different zones
- Load balancing: distribute traffic across instances

Goal

Design so that no single component failure brings down the whole system.



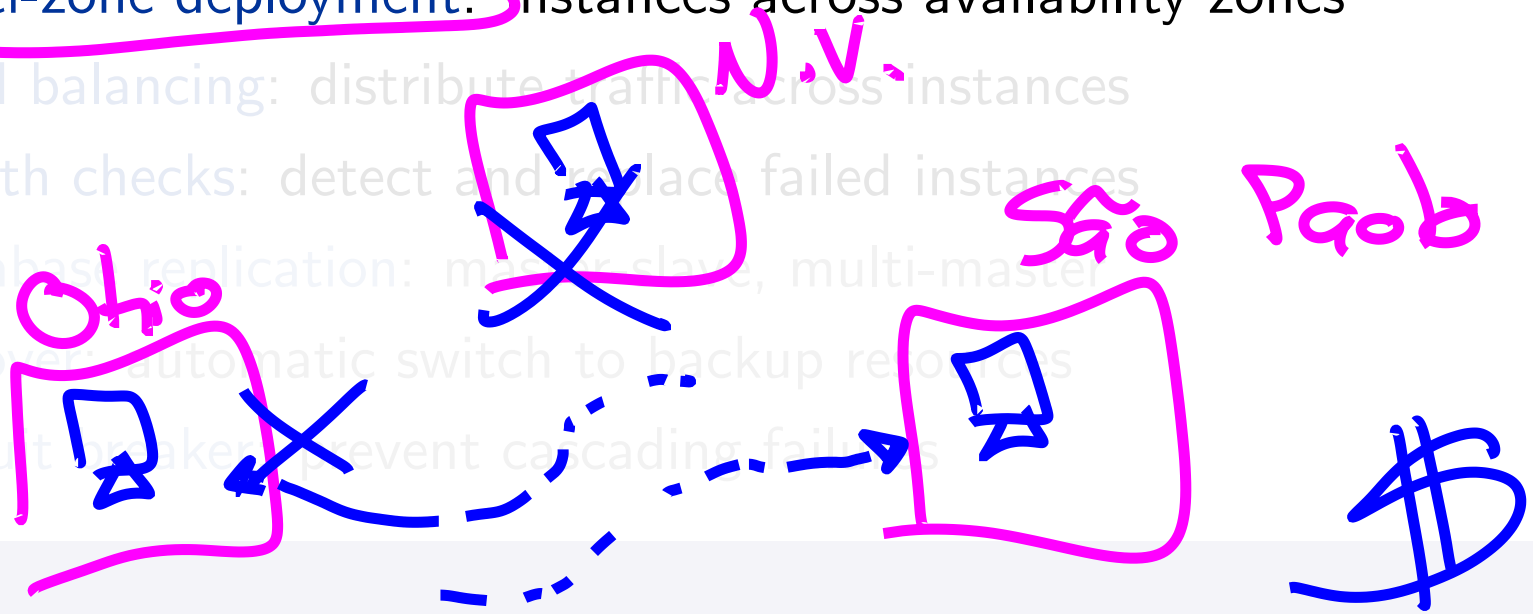
Achieving High Availability

Key Strategies

Redundancy at every layer: compute, storage, network, database.

Techniques:

- Multi-zone deployment: instances across availability zones
- Load balancing: distribute traffic across instances
- Health checks: detect and replace failed instances
- Database replication: master-slave, multi-master
- Failover: automatic switch to backup resources
- Circuit breakers: prevent cascading failures



Goal

Design so that no single component failure brings down the whole system.

Achieving High Availability

Key Strategies

Redundancy at every layer: compute, storage, network, database.

Techniques:

- Multi-zone deployment: instances across availability zones
- Load balancing: distribute traffic across instances
- Health checks: detect and replace failed instances
- Database replication: master-slave, multi-master
- Failover: automatic switch to backup resources
- Circuit breaker: prevent cascading failures

Goal

Design so that no single component failure brings down the whole system.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Achieving High Availability

Key Strategies

Redundancy at every layer: compute, storage, network, database.

Techniques:

- Multi-zone deployment: instances across availability zones
- Load balancing: distribute traffic across instances
- Health checks: detect and replace failed instances
- Database replication: master-slave, multi-master
- Failover: automatic switch to backup resources
- Circuit breaker: prevent cascading failures

Goal

Design so that no single component failure brings down the whole system.



Achieving High Availability

Key Strategies

Redundancy at every layer: compute, storage, network, database.

Techniques:

- Multi-zone deployment: instances across availability zones
- Load balancing: distribute traffic across instances
- Health checks: detect and replace failed instances
- Database replication: master-slave, multi-master
- Failover: automatic switch to backup instances
- Circuit breaker: prevent cascading failures

Goal

Design so that no single component failure brings down the whole system.





Achieving High Availability

Key Strategies

Redundancy at every layer: compute, storage, network, database.

Techniques:

- Multi-zone deployment: instances across availability zones
- Load balancing: distribute traffic across instances
- Health checks: detect and replace failed instances
- Database replication: master-slave, multi-master
- Failover: automatic switch to backup resources
- Circuit breaker: prevent cascading failures

Goal

Design so that no single component failure brings down the whole system.





Achieving High Availability

Key Strategies

Redundancy at every layer: compute, storage, network, database.

Techniques:

- Multi-zone deployment: instances across availability zones
- Load balancing: distribute traffic across instances
- Health checks: detect and replace failed instances
- Database replication: master-slave, multi-master
- Failover: automatic switch to backup resources
- Circuit breaker: prevent cascading failures

Goal

Design so that no single component failure brings down the whole system.





Achieving High Availability

Key Strategies

Redundancy at every layer: compute, storage, network, database.

Techniques:

- Multi-zone deployment: instances across availability zones
- Load balancing: distribute traffic across instances
- Health checks: detect and replace failed instances
- Database replication: master-slave, multi-master
- Failover: automatic switch to backup resources
- Circuit breaker: prevent cascading failures

Goal

Design so that no single component failure brings down the whole system.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Outline

1 High Availability & Resilience

2 Backup & Disaster Recovery

3 Monitoring & Observability

4 DevOps & Infrastructure as Code





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

RPO and RTO

RPO: Recovery Point Objective

- How much data can you lose?
- Last backup was 1 hour ago
- `RPO` = 1 hour
- Impacts business loss

RTO: Recovery Time Objective

- How fast must you recover?
- Takes 4 hours to restore
- `RTO` = 4 hours
- Impacts downtime cost

Define Before Disaster

Know your `RPO`/`RTO` *before* something breaks. Test recovery regularly.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

RPO and RTO

RPO: Recovery Point Objective

- How much data can you lose?
- Last backup was 1 hour ago
- `RPO = 1 hour`
- Impacts business loss

RTO: Recovery Time Objective

- How fast must you recover?
- Takes 4 hours to restore
- `RTO = 4 hours`
- Impacts downtime cost

Define Before Disaster

Know your `RPO/RTO` *before* something breaks. Test recovery regularly.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

RPO and RTO

RPO: Recovery Point Objective

- How much data can you lose?
- Last backup was 1 hour ago
- `RPO = 1 hour`
- Impacts business loss

RTO: Recovery Time Objective

- How fast must you recover?
- Takes 4 hours to restore
- `RTO = 4 hours`
- Impacts downtime cost

Define Before Disaster

Know your `RPO/RTO` *before* something breaks. Test recovery regularly



DR Strategy Types

Cold Standby

- No running backup
- Data backed up offline
- Slower recovery
- Lower cost
- Hours to days

Warm Standby

- Partial backup running
- Replicated data in sync
- Faster than cold
- Medium cost
- Minutes to hours

Hot/Active-Active

- Fully running backup
- Real-time replication
- Instant failover
- Highest cost
- Seconds or less





DR Strategy Types

Cold Standby

- No running backup
- Data backed up offline
- Slower recovery
- Lower cost
- Hours to days

Warm Standby

- Partial backup running
- Replicated data in sync
- Faster than cold
- Medium cost
- Minutes to hours

Hot/Active-Active

- Fully running backup
- Real-time replication
- Instant failover
- Highest cost
- Seconds or less

trade-off

- time
+ money





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

DR Strategy Types

Cold Standby

- No running backup
- Data backed up offline
- Slower recovery
- Lower cost
- Hours to days

Warm Standby

- Partial backup running
- Replicated data in sync
- Faster than cold
- Medium cost
- Minutes to hours

Hot/Active-Active

- Fully running backup
- Real-time replication
- Instant failover
- Highest cost
- Seconds or less



Disaster Recovery Strategies

Data

Increasing cost and complexity →

Cold Standby	Warm Standby	Hot Active-Active
Hours/days	*Minutes*	*Seconds*

Choice depends on business impact of downtime.





Disaster Recovery Strategies

Increasing cost and complexity →

Cold Standby	Warm Standby	Hot/Active-Active
Hours/days	*Minutes*	*Seconds*

Choice depends on business impact of downtime.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Outline

1 High Availability & Resilience

2 Backup & Disaster Recovery

3 Monitoring & Observability

4 DevOps & Infrastructure as Code





The Three Pillars

Metrics

Numeric measurements: CPU, memory, requests/sec, latency, error rate.

Time-series data.

Logs

Textual events: application messages, errors, warnings.

Searchable, indexed.

Traces

Request flow: which services handled which operations, where time was spent.

Use All Three

Metrics show *what*, logs show *why*, traces show *how*.





The Three Pillars

templates

Metrics

Numeric measurements: CPU, memory, requests/sec, latency, error rate.

Time-series data.

Logs

Textual events: application messages, errors, warnings.

Searchable, indexed.

query

Traces

Request flow: which services handled which operations, where time was spent.

Use All Three

Metrics show *what*, logs show *why*, traces show *how*.





The Three Pillars

Metrics

Numeric measurements: CPU, memory, requests/sec, latency, error rate.
Time-series data.

Logs

Textual events: application messages, errors, warnings.
Searchable, indexed.

Traces

Request flow: which services handled which operations, where time was spent.

Use All Three

Metrics show *what*, logs show *why*, traces show *how*.







UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

The Three Pillars

- retención

TTL → Time To Live

Metrics

Numeric measurements: CPU, memory, requests/sec, latency, error rate.
Time-series data.

Logs

Textual events: application messages, errors, warnings.
Searchable, indexed.

Traces

Request flow: which services handled which operations, where time was spent.

Use All Three

Metrics show *what*, logs show *why*, traces show *how*.





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

SLI, SLO, SLA

SLI
Service Level Indicator
Specific measurement (e.g., 99.5% uptime)

SLO
Service Level Objective
Target for SLI (e.g., "maintain 99.9%")

SLA
Service Level Agreement
Legal contract with penalties

Relationship

SLA is a contract. **SLO** is your internal target. **SLI** is what you measure.





SLI, SLO, SLA

SLI
Service Level Indicator
Specific measurement (e.g., 99.5% uptime)

SLO
Service Level Objective
Target for SLI (e.g., "maintain 99.9%")

SLA
Service Level Agreement
Legal contract with penalties

Relationship

SLA is a contract. **SLO** is your internal target. **SLI** is what you measure.





SLI, SLO, SLA

SLI
Service Level Indicator
Specific measurement (e.g., 99.5% uptime)

SLO
Service Level Objective
Target for SLI (e.g., "maintain 99.9%")

SLA
Service Level Agreement
Legal contract with penalties

Relationship

SLA is a contract. **SLO** is your internal target. **SLI** is what you measure.





SLI, SLO, SLA

SLI
Service Level Indicator
Specific measurement (e.g., 99.5% uptime)

SLO
Service Level Objective
Target for SLI (e.g., "maintain 99.9%")

SLA
Service Level Agreement
Legal contract with penalties

Relationship

SLA is a contract. **SLO** is your internal target. **SLI** is what you measure.



Outline

1 High Availability & Resilience

2 Backup & Disaster Recovery

3 Monitoring & Observability

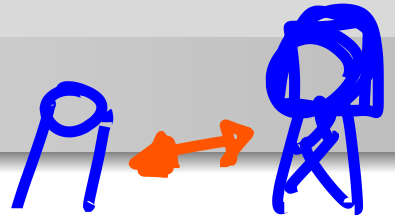
4 DevOps & Infrastructure as Code





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

DevOps Philosophy



Breaking Silos

Development and Operations work together, not against each other.
Shared responsibility for deployment, reliability, and performance.

Key Practices:

- Continuous Integration: code merged frequently, tested automatically
- Continuous Deployment: passing tests auto-deploy to production





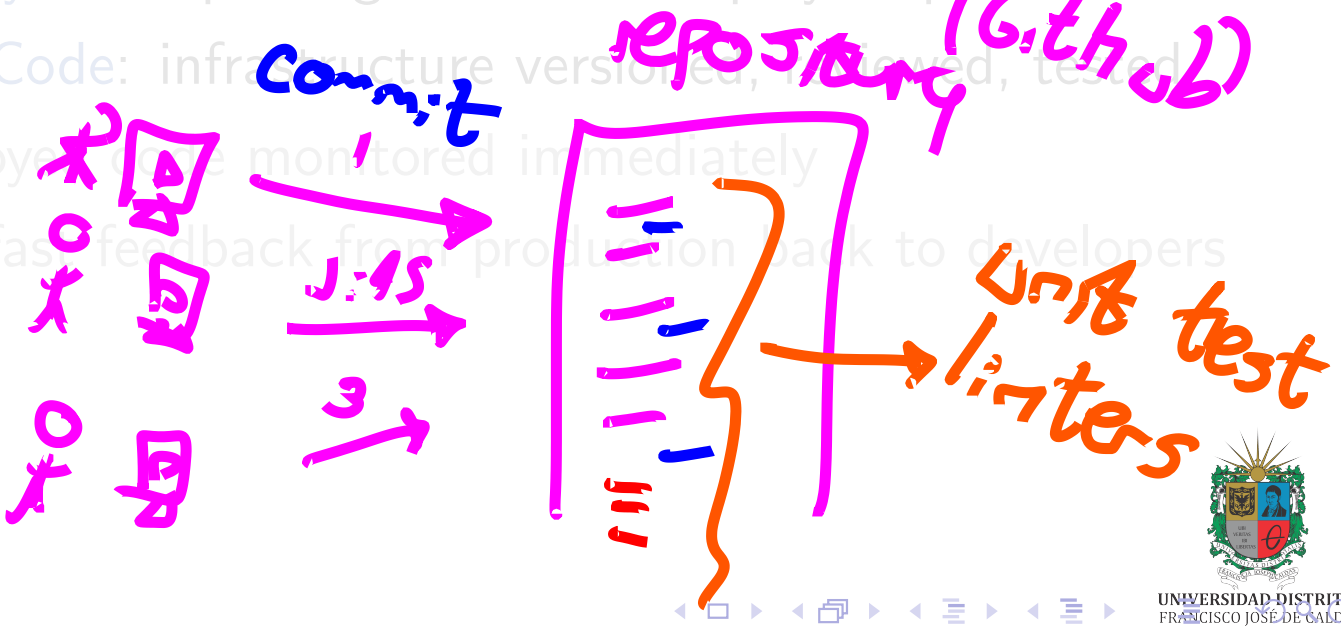
DevOps Philosophy

Breaking Silos

Development and Operations work together, not against each other. Shared responsibility for deployment, reliability, and performance.

Key Practices:

- Continuous Integration: code merged frequently, tested automatically
- Continuous Deployment: passing tests auto-deploy to production
- Infrastructure as Code: infrastructure versioned, reviewed, tested
- Monitoring: deployments are monitored immediately
- Feedback loops: fast feedback from production back to developers





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

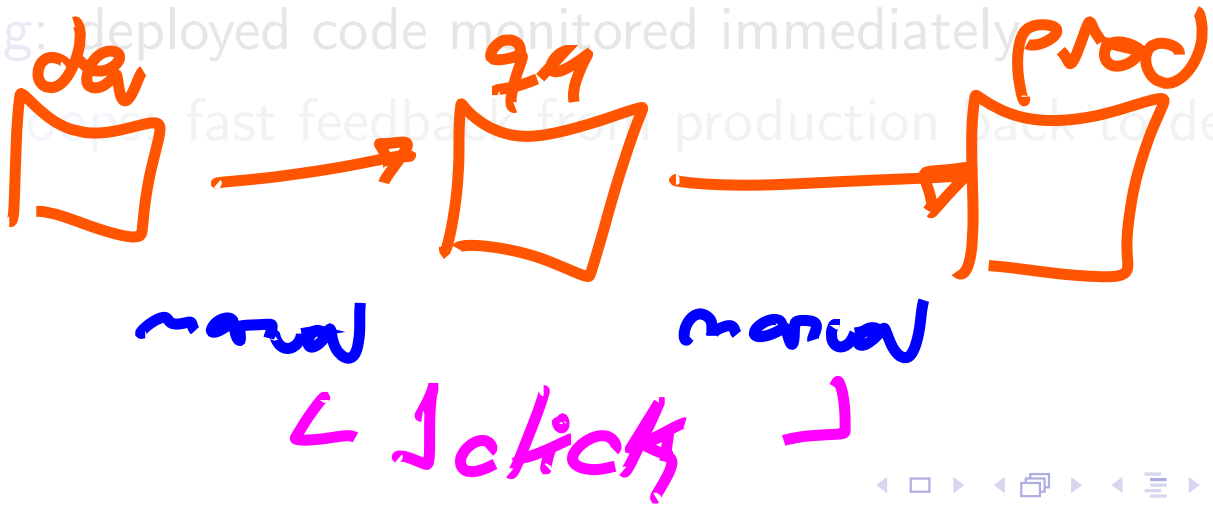
DevOps Philosophy

Breaking Silos

Development and Operations work together, not against each other. Shared responsibility for deployment, reliability, and performance.

Key Practices:

- Continuous Integration: code merged frequently, tested automatically
- Continuous Deployment: passing tests auto-deploy to production
- Infrastructure as Code: infrastructure versioned, reviewed, tested
- Monitoring: deployed code monitored immediately
- Feedback loops: fast feedback from production back to developers



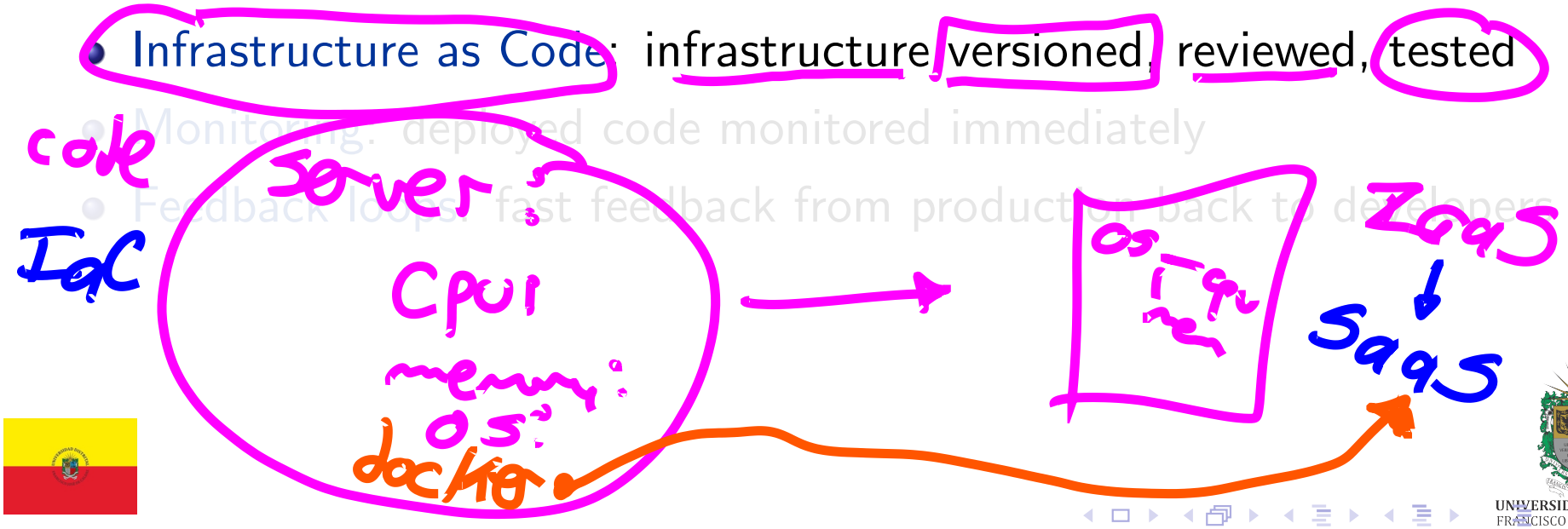
DevOps Philosophy

Breaking Silos

Development and Operations work together, not against each other. Shared responsibility for deployment, reliability, and performance.

Key Practices:

- Continuous Integration: code merged frequently, tested automatically
- Continuous Deployment: passing tests auto-deploy to production
- Infrastructure as Code: infrastructure versioned, reviewed, tested
- Monitoring: deployed code monitored immediately
- Feedback loops: fast feedback from production back to developers



DevOps Philosophy

Breaking Silos

Development and Operations work together, not against each other. Shared responsibility for deployment, reliability, and performance.

Key Practices:

- Continuous Integration: code merged frequently, tested automatically
- Continuous Deployment: passing tests auto-deploy to production
- Infrastructure as Code: infrastructure versioned, reviewed, tested
- Monitoring: deployed code monitored immediately
- Feedback loops: fast feedback from production back to developers

metrics
logs





DevOps Philosophy

Breaking Silos

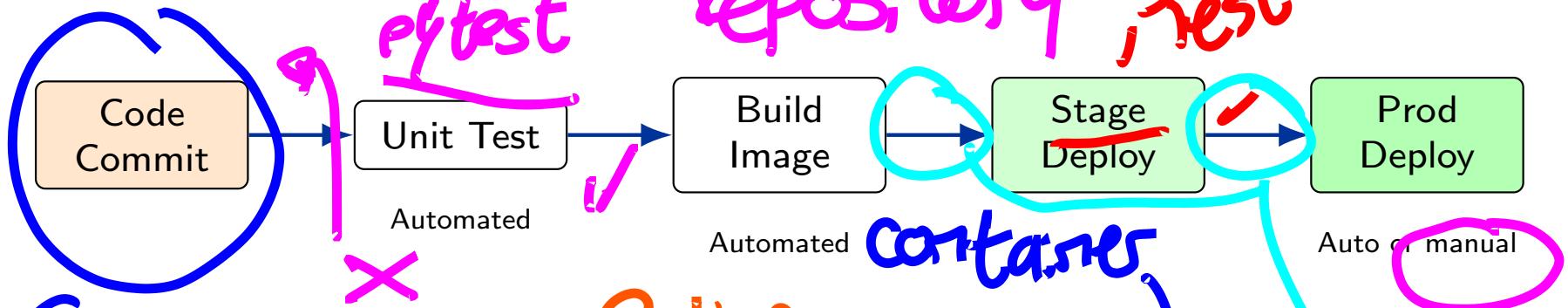
Development and Operations work together, not against each other. Shared responsibility for deployment, reliability, and performance.

Key Practices:

- Continuous Integration: code merged frequently, tested automatically
- Continuous Deployment: passing tests auto-deploy to production
- Infrastructure as Code: infrastructure versioned, reviewed, tested
- Monitoring: deployed code monitored immediately
- Feedback loops: fast feedback from production back to developers



CI/CD Pipeline



Git

code → git add . → git commit -m "..." → git push

Pull request → human review → senior → code quality assurement

containers → docker

CD Pipeline → Github Actions, Jenkins, Circle CI, Travis CI, Drone CI

Infrastructure as Code (IaC)

Principle

Infrastructure defined in `version-controlled` `code files`, not manual clicks. *Reproducible*, *testable*, *auditable*.

IaC v1

servers

IaC v2

serverless

X

Tools

Terraform, Ansible, CloudFormation, Pulumi, OpenTofu





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Infrastructure as Code (IaC)

Principle

Infrastructure defined in `version-controlled code files`, not manual clicks. *Reproducible, testable, auditable.*

Benefits:

- Reproducibility: rebuild infrastructure identically
- Version Control: `git history` of infrastructure changes
- Review Process: *code review* before infrastructure changes
- Testing: validate infrastructure before applying
- Documentation: `IaC` code is self-documenting

Tools

Terraform, Ansible, CloudFormation, Pulumi, OpenTofu





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Infrastructure as Code (IaC)

Principle

Infrastructure defined in `version-controlled code files`, not manual clicks. *Reproducible, testable, auditable.*

Benefits:

- Reproducibility: rebuild infrastructure identically
- Version Control: `git history` of infrastructure changes
- Review Process: *code review* before infrastructure changes
- Testing: validate infrastructure before applying
- Documentation: IaC code is self-documenting

Tools

Terraform, Ansible, CloudFormation, Pulumi, OpenTofu





Infrastructure as Code (IaC)

Principle

Infrastructure defined in `version-controlled code files`, not manual clicks. *Reproducible, testable, auditable.*

Benefits:

- Reproducibility: rebuild infrastructure identically
- Version Control: `git history` of infrastructure changes
- Review Process: *code review* before infrastructure changes
- Testing: validate infrastructure before applying
- Documentation: `IaC` code is self-documenting

Tools

Terraform, Ansible, CloudFormation, Pulumi, OpenTofu





Infrastructure as Code (IaC)

Principle

Infrastructure defined in `version-controlled code files`, not manual clicks. *Reproducible, testable, auditable.*

Benefits:

- Reproducibility: rebuild infrastructure identically
- Version Control: `git history` of infrastructure changes
- Review Process: *code review* before infrastructure changes
- Testing: validate infrastructure before applying
- Documentation: `IaC code is self-documenting`

Tools

Terraform, Ansible, CloudFormation, Pulumi, OpenTofu





Infrastructure as Code (IaC)

Principle

Infrastructure defined in `version-controlled code files`, not manual clicks. *Reproducible, testable, auditable.*

Benefits:

- Reproducibility: rebuild infrastructure identically
- Version Control: `git history` of infrastructure changes
- Review Process: *code review* before infrastructure changes
- Testing: validate infrastructure before applying
- Documentation: IaC code is self-documenting

artifacts

Tools

Terraform, Ansible, CloudFormation, Pulumi, OpenTofu



UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

Infrastructure as Code (IaC)

Principle

Infrastructure defined in `version-controlled code files`, not manual clicks. *Reproducible, testable, auditable.*

Benefits:

- Reproducibility: rebuild infrastructure identically
- Version Control: `git history` of infrastructure changes
- Review Process: *code review* before infrastructure changes
- Testing: validate infrastructure before applying
- Documentation: `IaC code` is self-documenting

Tools

Terraform, Ansible, CloudFormation, Pulumi, OpenTofu





Outline

1. High Availability & Resilience
2. Backup & Disaster Recovery
3. Monitoring & Observability
4. DevOps & Infrastructure as Code





Recap: Module 8

- **HA & Resilience**: redundancy at all layers, health checks, failover
- **Backup & DR**: define RPO/RTO, test recovery regularly
- **Monitoring**: metrics, logs, traces—all three matter
- **SLI/SLO/SLA**: measure performance, set targets, honor contracts
- **DevOps**: break silos between dev and ops
- **CI/CD**: automate testing and deployment
- **IaC**: version infrastructure like code





Thanks!

Questions?



Repo: *https://github.com/EngAndres/ud-public/tree/main/courses/cloud-computing*





UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS